SICAC

Sistema Integrado de Comitês para Avaliação Científica

Relatório técnico do sistema — backend e frontend

Gerado em 2026-11-10
Repositório: projeto_lab_prog — módulos backend/ e frontend/

1. Visão geral

O SICAC organiza a avaliação às cegas de projetos submetidos por discentes (via Sigaa) em edições anuais de uma mostra/edital científico, do tipo SEMITI/SEMIC da UFMA — evento que acontece simultaneamente em vários campi. Avaliadores (professores) recebem projetos atribuídos automaticamente dentro do mesmo comitê de avaliação *e* do mesmo campus de atuação, nunca veem quem são os autores ou orientadores, e dão nota por critério configurável. Administradores gerenciam edições, critérios, atribuições e a consolidação dos resultados.

O sistema é dividido em dois módulos independentes neste repositório: um backend em Spring Boot (Java 21) que expõe a API REST, e um frontend em React + TypeScript que consome essa API. Não há mais dados simulados ("mock") em nenhuma das duas pontas — cada tela do frontend chama o endpoint real correspondente.

1.1 Arquitetura e stack

Camada	Tecnologia	Observações
Backend	Java 21, Spring Boot 3.3.4, Spring Security 6, Spring Data JPA	Auth por JWT, autorização por role via SecurityFilterChain
Banco de dados	PostgreSQL 16 + Flyway	7 migrations versionadas (V1 schema, V2 seed, V3-V7 evolução comitê/campus — ver seção 12)
Testes (backend)	JUnit 5, Mockito, AssertJ, H2 (perfil de teste), MockMvc	208 testes — serviço, repository e controller
Frontend	React 19, TypeScript, Vite, React Router, TanStack Query	CSS próprio (sem biblioteca de componentes), sem dados mock
Orquestração	Docker Compose (Postgres) + Maven (backend) + npm (frontend)	O serviço backend do docker-compose ainda não builda — falta Dockerfile

1.2 Estrutura do repositório

```
projeto_lab_prog/
|-- docker-compose.yml   # sobe banco (+ backend/frontend)
|-- docs/                # modelagem, plano de implementação
|-- diagrams/        # diagramas de casos de uso e classes
|-- backend/         # API Spring Boot — 20 entidades, serviços, controllers, testes
`-- frontend/        # React + TypeScript, integrado de verdade com a API
```

2. Modelo de dados — 20 entidades

As entidades (`backend/.../entity/`) estão organizadas em cinco grupos. **Comitê** e **Campus** substituem o que antes era só "área de conhecimento" — ver a seção 12 para a motivação e o histórico dessa mudança.

2.1 Pessoas e acesso

- **Pessoa** — identificação de qualquer humano no sistema (nome, CPF único, matrícula, SIAPE, e-mail único, instituição). Separada de Usuario de propósito: autores/orientadores também são Pessoa, mesmo sem login, para permitir a checagem de conflito de interesse.
- **Usuario** — a conta que loga. Tem pessoa (1:1), senha com hash, *tipo* (a role), status de ativação (PENDENTE_ATIVACAO / ATIVO / INATIVO), *precisaTrocarSenha* (força a troca de senha no próximo acesso — ver seção 13) e, se for avaliador, um **Comitê** fixo — vale pra todas as edições em que participar, não muda a cada edição.
- **Instituicao** — nome e sigla; referenciada por Pessoa e Projeto.
- **Comitê** — nome e sigla; é o comitê de avaliação (ex.: "Comitê de Tecnologia") que casa avaliador com projeto no sorteio. Cada projeto responde a exatamente um comitê, e cada avaliador pertence a exatamente um comitê — por isso não é mais uma tabela de junção N:N, é FK direta em Usuario e em Projeto.
- **ComiteAlias** — mapeia um texto cru vindo de fora (ex.: "Engenharias", como chega do Sigaa) para um Comitê real.
- **Campus** — nome e sigla; o local físico do evento (ex.: "Campus Bacanga"), separado de Instituição porque o mesmo evento pode acontecer em vários campi da mesma instituição.
- **CampusAlias** — mesmo papel do ComiteAlias, para strings cruas de campus vindas de importação.

2.2 Edição (o ciclo anual)

- **Edicao** — nome, ano, status (PLANEJADA / ATIVA / ENCERRADA), janela de datas, mínimo de avaliadores por projeto, limite de projetos por avaliador, divergência aceitável e escala de nota.
- **Criterio** — nome, peso e ordem, pertence a uma Edicao; compõe a nota final ponderada.
- **EdicaoAvaliador** — junção Edicao × Usuario: se o avaliador está ativo *nesta* edição, com limite de carga próprio e o **Campus** em que atua *nesta* edição especificamente — diferente do comitê, o campus pode mudar de uma edição para outra.

2.3 Projetos e autoria

- **Projeto** — o que é avaliado: identificador do Sigaa (único por edição), título, conteúdo técnico, comitê, campus de origem, instituição e status (SUBMETIDO / EM_AVALIACAO / AVALIADO).
- **ProjetoDiscente** — Pessoa × Projeto com papel (AUTOR ou COAUTOR); só um AUTOR por projeto.
- **ProjetoOrientador** — Pessoa × Projeto, sempre como orientador. Autores, coautores e orientadores juntos formam a base do conflito de interesse checado no sorteio.

2.4 Atribuição e avaliação

- **Atribuicao** — liga um avaliador a um projeto; status (ATIVA/CANCELADA), origem (AUTOMATICA/MANUAL) e motivo (quando manual). Só uma ATIVA por par projeto/avaliador.
- **Avaliacao** — 1:1 com Atribuicao; status (RASCUNHO/ENVIADA), nota final calculada pelo servidor (nunca aceita como input direto), e os campos de reabertura.
- **AvaliacaoNota** — Avaliacao × Criterio → a nota naquele critério; é a partir daqui que a nota final é calculada.

2.5 Importação e auditoria

- **ImportacaoLote** e **ImportacaoItem** — modelam um lote de importação (projetos do Sigaa ou avaliadores por arquivo) e cada item dentro dele, com status de processamento. Nenhum controller usa essas entidades ainda — a importação de verdade não foi implementada.
- **IntegracaoCredencial** — credencial de uma integração externa (hash da chave, nunca o valor em texto puro). Reservada para quando a importação existir.
- **LogAuditoria** — quem fez o quê: usuário, ação, entidade afetada, detalhes e data/hora. Toda ação relevante do admin é registrada aqui.

3. Onde está a role do usuário

A role não é uma entidade própria — é um campo (`Usuario.tipo`, só ADMIN ou AVALIADOR) que atravessa três camadas até chegar na autorização de cada rota:

- **Entidade:** coluna `usuario.tipo` no schema (enum UsuarioTipo).
- **Login (JWT):** JwtService grava *tipo* como claim do token; UsuarioPrincipal traduz isso para uma authority do Spring Security — "ROLE_" + tipo (ex.: ROLE_ADMIN, ROLE_AVALIADOR).
- **Autorização de rota:** SecurityConfig decide por prefixo de URL — /api/admin/** exige ROLE_ADMIN, /api/avaliador/** exige ROLE_AVALIADOR. Não há checagem método a método.

O frontend não re-decide nada disso: apenas decodifica o claim *tipo* do JWT para redirecionar quem tenta acessar a área errada. É só UX — a segurança de fato está no backend.

4. Serviços do backend

- **EdicaoService** — cria, ativa e encerra edições. Só uma edição ativa por vez; uma edição encerrada não volta a ser ativada.
- **CriterioService** — cria e remove critérios de uma edição (peso > 0; edição encerrada é somente leitura) e soma os pesos.
- **AtribuicaoService** — o sorteio automático: sorteia até o mínimo da edição, só entre avaliadores ativos na edição, no mesmo comitê *e* no mesmo campus do projeto, sem conflito de interesse e respeitando o limite de carga. Se o pool não bastar, atribui o que der e devolve os "faltantes" em vez de falhar silenciosamente. A reatribuição manual passa pelas mesmas regras de comitê/campus — elas valem tanto pro sorteio automático quanto pra escolha manual do admin (ver seção 12).
- **AvaliacaoService** — o que o avaliador faz: só vê os projetos atribuídos a ele, em visão cega (nunca autor, orientador ou instituição). Salva rascunho, envia (todos os critérios obrigatórios, nota na escala, dentro do prazo) e calcula a nota final como média ponderada. Só o admin reabre uma avaliação enviada.
- **ConsolidacaoService** — calcula a nota do projeto, a divergência entre avaliadores e se o projeto já está avaliado, atualizando o status do projeto.
- **CatalogoService** — ponto único de "buscar ou criar" comitê, campus e instituição a partir de um nome digitado: primeiro tenta um alias conhecido, senão busca pelo nome (sem diferenciar maiúscula/minúscula), senão cria. Usado por ProjetoService, UsuarioService e EdicaoAvaliadorService — evita que o mesmo nome escrito de formas diferentes crie registros duplicados.
- **AuthService** — login (JWT) e troca de senha (*trocarSenha*: valida a senha atual, re-hasheia a nova, limpa *precisaTrocarSenha* e devolve um token novo — ver seção 13).
- **UsuarioService** — cadastro de avaliador pelo admin: cria Pessoa + Usuario já com a senha padrão (configurável) e *precisaTrocarSenha* = true, sem token nem link de ativação; reaproveita a instituição pelo nome se já existir. Dispara EmailService com a senha — hoje só loga no console (ConsoleEmailService), sem provedor de e-mail real. Também lista, atualiza e remove avaliadores.
- **ProjetoService** — cadastro manual de projeto (não é a importação do Sigaa), além de listar, atualizar e remover. Recusa edição encerrada e sigaaId duplicado na mesma edição (gera um próprio se não vier nenhum); reaproveita comitê/campus/instituição pelo nome (via `CatalogoService`) e a pessoa do autor pelo CPF.
- **EdicaoAvaliadorService** — vincula um avaliador a uma edição, fixando o comitê do avaliador (usuario.comite) e o campus em que ele atua nesta edição especificamente. Os dois são obrigatórios: o banco rejeita um avaliador ativo sem os dois definidos, então o serviço valida antes de tentar salvar. Um avaliador só fica ativo em uma edição por vez: vincular a uma nova libera (INATIVO) qualquer outra em que ele estivesse ativo, para o sorteio automático só considerar quem está livre.

- **AuditoriaService** — grava o log de cada ação relevante.

5. Endpoints da API

Todas as rotas autenticadas usam o usuário logado via JWT; as de admin exigem ROLE_ADMIN e as do avaliador exigem ROLE_AVALIADOR.

Rota	Quem	O que faz
`POST /auth/login` `PUT /api/conta/senha`	público autenticado	Login; troca a própria senha (devolve token novo)
`GET/POST /api/admin/avaliadores` `PUT/DELETE .../{id}`	admin	Lista, cadastra (nasce com a senha padrão, devolvida na resposta), atualiza e remove avaliador
`POST /api/admin/avaliadores/{id}/edicoes`	admin	Vincula o avaliador a uma edição, com comitê e campus obrigatórios; libera qualquer outra edição ativa anterior
`GET/POST /api/admin/projetos` `PUT/DELETE .../{id}`	admin	Lista, cadastra (manual, não é importação), atualiza e remove projeto
`GET/POST /api/admin/edicoes` `GET /api/admin/edicoes/{id}`	admin	Lista, cria e busca edições
`POST /api/admin/edicoes/{id}/ativar` `POST /api/admin/edicoes/{id}/encerrar`	admin	Muda o status da edição
`GET/POST /api/admin/edicoes/{id}/criterios` `DELETE /api/admin/criterios/{id}`	admin	Critérios da edição
`POST /api/admin/atribuicoes/projetos/{id}` `POST /api/admin/atribuicoes/edicao-ativa`	admin	Sorteio automático de um projeto ou de toda a edição ativa
`POST /api/admin/atribuicoes/{id}/redistribuir`	admin	Troca um avaliador, com motivo obrigatório
`GET /api/admin/projetos/{id}/consolidacao`	admin	Nota do projeto, divergência e comparação entre avaliadores
`POST /api/admin/avaliacoes/{id}/reabrir`	admin	Reabre uma avaliação enviada, com motivo
`GET /api/avaliador/projetos`	avaliador	Projetos atribuídos, em visão cega
`GET /api/avaliador/atribuicoes/{id}`	avaliador	Formulário da avaliação e notas já salvas
`PUT /api/avaliador/atribuicoes/{id}/rascunho`	avaliador	Salva notas parciais
`POST /api/avaliador/atribuicoes/{id}/enviar`	avaliador	Envio final da avaliação

Ficaram de fora deste recorte: a importação de verdade de projetos do Sigaa, o envio de e-mail real (hoje só loga no console), os endpoints de listagem de Atribuições e Consolidação (por isso essas duas telas do frontend continuam sendo formulário por ID), e a listagem/seed de comitê/campus/instituição prontos pra escolher — o cadastro de avaliador, o de projeto e o vínculo a uma edição aceitam apenas o nome digitado, que é reaproveitado se já existir (via CatalogoService).

6. Testes automatizados

São 208 testes, em três camadas, executados com `mvn test`:

Camada	Qtde	O que cobre
Serviço (Mockito puro, sem banco)	150	AtribuicaoServiceTest (24), AvaliacaoServiceTest (23), ConsolidacaoServiceTest (17), EdicaoServiceTest (18), ProjetoServiceTest (14), UsuarioServiceTest (14), CriterioServiceTest (12), CatalogoServiceTest (12), EdicaoAvaliadorServiceTest (9), AuthServiceTest (5), AuditoriaServiceTest (2). Conflito de interesse, limite de carga, pool insuficiente, elegibilidade por comitê *e* campus (automática e na reatribuição manual), acesso cruzado entre avaliadores, prazo, nota fora da escala, média ponderada, divergência, duplicidade de e-mail/CPF/sigaaId, que vincular a uma nova edição libera a anterior, troca de senha (senha atual incorreta, usuário inexistente, token novo devolvido), e a resolução de comitê/campus/instituição (alias, reaproveitamento, criação) isolada no CatalogoService.
Repository (H2 em memória)	8	UsuarioRepositoryTest e MapeamentoObjetoRelacionalTest — CPF único, sigaaId único por edição, o CHECK que exige comitê de um avaliador ativo, e os defaults do schema, contra uma cópia adaptada das 9 migrations (sem TIMESTAMPTZ, índice único parcial nem expressão em índice, que o H2 não entende do mesmo jeito que o Postgres).
Controller (@WebMvcTest + MockMvc)	50	Um arquivo por controller, com autenticação injetada direto (sem passar pelo filtro JWT real). Cobrem status HTTP, validação de @Valid (inclusive comitê/campus obrigatórios no vínculo, e a senha nova com no mínimo 8 caracteres em ContaControllerTest), e que cada rota exige o papel certo (401 sem autenticação, 403 com papel errado).

Quebrar de propósito uma regra (o filtro de conflito de interesse, a validação de um controller) faz o teste correspondente falhar — são testes que pegam regressão de verdade, não só cobertura de linha.

7. Frontend

Interface em React + TypeScript integrada de verdade com a API — sem dados simulados. Rotas protegidas por papel (/admin/* exige ADMIN, /avaliador/* exige AVALIADOR), cache e mutações via TanStack Query, visual com CSS próprio (fundo cinza claro, cards, tipografia Inter).

7.1 Telas

Área	Telas	Situação
Autenticação	Login, Trocar senha	Reais
Admin	Edições, Critérios, Avaliadores, Projetos, Atribuições, Consolidação	Reais
Admin	Importação	Placeholder — formulário existe, não chama API (Sigaa não implementado)
Avaliador	Meus projetos, Avaliação (rascunho/envio)	Reais

7.2 Lacunas conhecidas

- **Atribuições** e **Consolidação** são formulários por ID, não listas navegáveis — o backend não expõe endpoint de listagem pra esses dois recursos (Avaliadores e Projetos já são listas de verdade, ver seção 7.1).
- **Consolidação** mostra a nota final por avaliador, mas não o detalhe por critério (o endpoint não devolve isso), e reabrir avaliação pede um ID que a própria tela não lista.
- Não há envio de e-mail real: a senha padrão do avaliador recém-cadastrado aparece na própria resposta do cadastro, para ser repassada manualmente (ver seção 13).

A tela **Avaliadores** tem a seção "Vincular a uma edição" (`POST /api/admin/avaliadores/{id}/edicoes`, resolvido numa revisão anterior — ver seção 11), já enviando comitê e campus (a lacuna registrada na seção 12.6 de uma revisão anterior, de o frontend ainda falar a língua antiga de "área", já não existe mais — ver seção 13).

8. Ciclo completo validado de ponta a ponta

O fluxo abaixo foi testado contra a API real (Postgres, não H2), não apenas camada por camada:

- Cadastrar avaliador (`POST /api/admin/avaliadores`) → avaliador loga com a senha padrão devolvida na resposta → troca a senha em `/trocar-senha`, forçada automaticamente antes de acessar qualquer outra tela (ver seção 13).
- Criar edição, ativar, adicionar critério.
- Vincular o avaliador à edição ativa com comitê e campus (`POST /api/admin/avaliadores/{id}/edicoes`).
- Cadastrar um projeto na mesma edição, comitê e campus (`POST /api/admin/projetos`).
- Sortear avaliadores para o projeto — encontrou o avaliador vinculado.
- Logar como avaliador, ver o projeto em "Meus projetos", enviar a avaliação.
- Admin consultar a consolidação e ver a nota final.

Antes da primeira rodada de implementação, nenhuma dessas etapas (exceto login/edição) era possível sem inserir dados manualmente no Postgres — o ciclo inteiro passa pela API desde então. Nesta revisão (comitê/campus, seção 12), o passo de vínculo com os campos novos foi reconfirmado por chamada HTTP direta contra o Postgres de dev, incluindo a rejeição com 400 quando falta campus; os demais passos continuam cobertos pela suíte automatizada (seção 6), mas não foram repetidos manualmente nesta rodada.

9. Pontos de atenção técnica (revisão de código)

Achados de uma revisão de código sobre o estado de backend e frontend numa revisão anterior a esta, por categoria. **Os 10 itens abaixo foram corrigidos** na mesma sessão em que foram encontrados — ver a seção 11 para o que mudou em cada um. A seção 12 documenta a revisão mais recente, focada no banco de dados e na modelagem de comitê/campus.

9.1 Correção / comportamento

- **403 ambíguo:** o cliente HTTP do frontend trata qualquer 403 com token presente como sessão expirada e desloga o usuário. Mas o backend também retorna 403 para uma negação de acesso legítima (ex.: avaliador tentando abrir uma atribuição que não é dele), já que um token inválido/expirado não gera 401 — vira 403 pelo filtro de segurança. Hoje os dois casos são indistinguíveis no front.
- **AreaConhecimentoAlias sem uso:** a única mecânica de alias/normalização de nomes de área existe no modelo mas não é chamada em nenhum lugar. Duas grafias diferentes da mesma área (ex.: "Computação" vs. "Ciência da Computação") criam duas linhas distintas, fragmentando o pool usado no sorteio automático.
- **Vínculo avaliador↔edição sem UI:** o endpoint existe no backend, mas nenhuma tela do frontend o chama — sem isso, o sorteio nunca encontra ninguém usando só a interface (já documentado no README do frontend).

9.2 Eficiência

- **ConsolidacaoService.consolidar()** faz uma consulta por avaliador atribuído em vez de uma única consulta em lote — o custo cresce linearmente com o número de avaliadores por projeto.
- **AtribuicaoService:** o sorteio de toda a edição ativa reconsulta o pool de avaliadores a cada projeto (mesmo sendo igual entre eles) e consulta a área de cada avaliador candidato individualmente em vez de em lote.
- **AvaliacaoService.detalhar()** busca a mesma Avaliacao duas vezes na mesma chamada — uma direta e outra dentro de um método auxiliar que repete a consulta.

9.3 Duplicação / simplificação

- A lógica de "buscar ou criar" área/instituição pelo nome está reimplementada de forma quase idêntica em três serviços (ProjetoService, UsuarioService, EdicaoAvaliadorService).
- A checagem "edição encerrada é somente leitura" está copiada, idêntica, em três serviços diferentes.
- O frontend recalcula a fórmula de nota final (média ponderada) no navegador, duplicando a lógica do backend em JavaScript — se a fórmula do servidor mudar, a pré-visualização pode ficar desatualizada silenciosamente.
- O mapeamento "papel → rota inicial" (ADMIN → /admin/edicoes, AVALIADOR → /avaliador/projetos) está repetido em quatro lugares do frontend em vez de centralizado numa função.

Nenhum destes pontos quebra o fluxo hoje — são oportunidades de robustez e manutenção, não bugs bloqueantes.

10. Como rodar

10.1 Banco de dados

`docker compose up -d db` — sobe o Postgres na porta 5433 do host.

10.2 Backend

`cd backend && mvn spring-boot:run` — porta 8080; Flyway aplica o schema e semeia o admin automaticamente.

10.3 Frontend

`cd frontend && npm install && npm run dev` — porta 5173; chama a API em localhost:8080 por padrão.

10.4 Usuário de teste

Perfil	E-mail	Senha
Administrador	admin@instituicao.edu.br	admin123

Não existe usuário avaliador por padrão — é cadastrado pelo admin em tempo de execução, pela tela "Avaliadores" ou diretamente via `POST /api/admin/avaliadores`, e já nasce com a senha padrão `Trocar@123` (ver seção 13).

11. Correções aplicadas na revisão anterior

Os 10 achados da seção 9 foram corrigidos na mesma sessão em que foram encontrados, com testes atualizados e validação de ponta a ponta contra a API real (Postgres) depois de cada mudança. A suíte foi de 173 para 179 testes nessa época — ver seção 6 para a contagem atual, depois da revisão de comitê/campus (seção 12).

11.1 Correção / comportamento

- **401 x 403:** `SecurityConfig` agora define um `AuthenticationEntryPoint` e um `AccessDeniedHandler` próprios — sem token ou com token inválido/expirado dá 401; autenticado com o papel errado dá 403. O frontend (`client.ts`) só desloga no 401; um 403 aparece como erro normal na tela, sem deslogar. `WebSecurityTestConfig` replica a mesma distinção, e os testes "sem autenticação" passaram a esperar 401 em vez de 403.
- **Alias de área ligado:** novo `CatalogoService` resolve área/instituição a partir do nome digitado — para área, primeiro consulta `AreaConhecimentoAlias` (antes nunca usado); só cria uma área nova se não achar nem alias nem nome exato. Usado por `UsuarioService`, `ProjetoService` e `EdicaoAvaliadorService`.
- **Vínculo avaliador↔edição:** a tela **Avaliadores** ganhou a seção "Vincular a uma edição", que chama o endpoint que já existia no backend. Agora o ciclo completo (cadastro → vínculo → sorteio → avaliação → consolidação) é navegável inteiramente pela interface, sem precisar chamar a API na mão.

11.2 Eficiência

- `ConsolidacaoService.consolidar()` passou a buscar todas as avaliações do projeto em uma única consulta (`findByAtribuicaoIdIn`) em vez de uma por avaliador.
- O sorteio automático (`AtribuicaoService`) agora busca o pool de avaliadores da edição uma única vez por chamada a "sortear edição ativa" (reaproveitado entre projetos) e busca a área de todos os candidatos de uma vez (`findByAvaliadorIdIn`) em vez de um avaliador por vez.
- `AvaliacaoService.detalhar()` passou a buscar a avaliação uma única vez e reaproveitar o resultado, em vez de buscar de novo dentro de um método auxiliar.

11.3 Duplicação / simplificação

- A checagem "edição encerrada é somente leitura" foi centralizada em `EdicaoService.garantirEditavel(edicao)`, chamada por `CriterioService`, `ProjetoService` e `EdicaoAvaliadorService` em vez de cada um reimplementar a mesma condição.
- O frontend deixou de recalcular a nota final no navegador antes do envio — a nota final só aparece depois que o servidor a calcula de verdade, eliminando a duplicação da fórmula (e o risco de ela ficar desatualizada se a fórmula do backend mudar).
- O mapeamento "papel → rota inicial" foi centralizado em `rotaInicial()` (`frontend/src/auth/session.ts`), usado pelos quatro lugares que antes repetiam o mesmo ternário.

11.4 Bug encontrado durante a validação

Ao testar o novo 401/403 contra a API real, as respostas de erro escritas à mão (o `AuthenticationEntryPoint` e o `AccessDeniedHandler` novos) saíam em ISO-8859-1 em vez de UTF-8 — acentos como "ç" e "ã" chegavam como bytes de um byte só, e um cliente que decodifica como UTF-8 (como o `fetch` do navegador) exibiria caracteres errados. Corrigido declarando `charset=UTF-8` explicitamente no `Content-Type` dessas duas respostas; confirmado byte a byte depois.

12. Revisão de banco e modelagem de comitê/campus

Revisão mais recente, motivada por um requisito novo: o evento acontece em vários campi simultaneamente (modelo real pesquisado: SEMITI/SEMIC da UFMA), e um avaliador só pode avaliar projetos do seu próprio comitê *e* do seu próprio campus — ex.: um avaliador do Campus Bacanga só avalia projetos do Comitê de Tecnologia que também sejam do Campus Bacanga. Também veio a confirmação de que a atribuição, na prática, tende a ser manual (o admin escolhe), não um sorteio automático disparado em lote — mas as mesmas regras de elegibilidade (comitê, campus, conflito de interesse, limite de carga) continuam valendo nos dois casos; o sorteio automático foi mantido no backend, não removido.

12.1 Decisões de modelagem

- **Área de conhecimento virou Comitê:** renomeação direta — cada projeto responde a exatamente um comitê, igual já era com área.
- **Comitê do avaliador é fixo, não por edição:** um avaliador pertence a um único comitê, que vale em todas as edições em que participar — por isso é FK direta em Usuario, não mais uma tabela de junção N:N (AvaliadorArea foi removida).
- **Campus é por edição:** o mesmo avaliador pode atuar num campus numa edição e noutro na seguinte — por isso mora em EdicaoAvaliador, ao lado do comitê fixo em Usuario. É uma assimetria intencional entre os dois, não um descuido.
- **Instituição foi mantida separada de Campus:** o mesmo evento acontece em vários campi da mesma instituição (UFMA), então os dois conceitos não se confundem.

12.2 Migrations aplicadas (V3 a V7)

Migration	O que faz
V3 — comitê e local	Renomeia área→comitê em todas as tabelas envolvidas; cria a tabela "local" (nome provisório) e as colunas de FK em projeto e edicao_avaliador.
V4 — renomeia local para campus	"Local" era nome provisório enquanto a modelagem era decidida; renomeado pra "campus", nome final.
V5 — comitê único e uniques	Move o comitê do avaliador da tabela de junção (avaliador_comite, sem escopo de edição) para uma coluna direta em usuario, migrando os dados existentes antes do DROP TABLE. Adiciona UNIQUE em comite.nome, campus.nome, instituicao.nome e pessoa.email.
V6 — fecha inconsistências comitê/campus	Troca as UNIQUE da V5 por índice sobre lower(nome) (a V5 era case-sensitive, mas o CatalogoService resolve ignorando maiúscula/minúscula). Remove a coluna de hierarquia comite_pai_id (herdada de área, sem uso no código). Adiciona CHECK exigindo comitê de todo avaliador ATIVO. Adiciona comite.sigla e a tabela campus_alias (espelhando comite_alias).
V7 — exige campus de avaliador ativo	CHECK exigindo campus de todo vínculo avaliador-edição ATIVO — mesmo princípio da V6, agora para campus.

Cada migration foi validada contra uma cópia do Postgres de desenvolvimento já com dados reais (não só um banco vazio), e contra a cópia adaptada usada pelos testes em H2, antes de ser considerada pronta.

12.3 Revisão de inconsistências do banco

Pedido explícito de revisão do schema resultante, em duas rodadas, encontrou e corrigiu:

- UNIQUE case-sensitive em nome de comitê/campus/instituição, que não batia com a resolução case-insensitive do CatalogoService — trocado por índice sobre lower(nome) (V6).
- Coluna comite_pai_id sem nenhum uso no código, herdada de área de conhecimento sem que a hierarquia fizesse sentido para comitê de avaliação — removida (V6).

- Nenhuma restrição ligava avaliador ativo a ter comitê e campus definidos — um avaliador podia ficar silenciosamente inelegível pra tudo, sem erro nenhum. Fechado com CHECK constraints (V6, V7); dois vínculos de teste que já existiam sem campus foram corrigidos manualmente antes da V7 poder ser aplicada.
- Nomes de constraint e sequence que sobraram da renomeação (ex.: local_pkey, projeto_area_id_fkey) foram corrigidos pra bater com os nomes novos — cosmético, mas deixa o schema legível.
- Faltava campus_alias (equivalente a comite_alias) para resolver string crua de campus vinda de importação — criada (V6).

12.4 Refatoração do backend

A camada Java foi inteiramente atualizada pra bater com o schema novo — não só renomeada:

- Entidades: Comite e ComiteAlias substituem AreaConhecimento/AreaConhecimentoAlias; Campus e CampusAlias são novas; AvaliadorArea foi removida; Usuario ganhou comite, EdicaoAvaliador e Projeto ganharam campus.
- CatalogoService ganhou resolverCampus, no mesmo padrão de resolverComite/resolverInstituicao.
- AtribuicaoService: a elegibilidade, que buscava a área do avaliador numa tabela de junção, agora compara direto usuario.comite e participacao.campus contra o projeto — mais simples, e passou a casar por comitê *e* campus. A mesma checagem foi adicionada na reatribuição manual.
- EdicaoAvaliadorService.vincular: comitê e campus passaram de opcionais a obrigatórios, validados antes de salvar — evita que o CHECK do banco rejeite com um erro de SQL em vez de uma mensagem clara.
- 7 arquivos de teste atualizados pro modelo novo, incluindo casos novos para a elegibilidade por campus; suíte validada de ponta a ponta contra o Postgres de dev (login, listagem e o fluxo de vínculo com os campos novos).

12.5 Dados de exemplo

O banco de desenvolvimento foi repovoado do zero com um conjunto coerente cobrindo as 20 tabelas: 2 instituições, 4 campi e 5 comitês (nomes inspirados na estrutura real pesquisada do SEMITI/UFMA), 12 pessoas / 7 usuários (um deles pendente de ativação, sem comitê, pra exercitar o CHECK), 3 edições, 10 projetos (todos com comitê, campus e instituição preenchidos), 10 atribuições manuais casadas por comitê, 7 avaliações e lotes de importação de exemplo com item de sucesso, erro, duplicado e pendente de mapeamento. O login do admin padrão (seção 10.4) continua funcionando.

12.6 Gap conhecido: frontend desatualizado (resolvido)

O frontend **não foi tocado** nesta revisão e, na época, ainda falava a língua antiga ("área", `areaNome`) em AvaliadoresPage, ProjetosPage e nas telas do avaliador. Esse gap foi fechado numa revisão posterior, fora do escopo registrado aqui: hoje o frontend inteiro já fala comitê/campus (nenhuma referência a `areaNome` ou `AreaConhecimento` resta em código, backend ou frontend) — ver seção 7.

13. Revisão de autenticação: senha padrão e troca obrigatória

Revisão mais recente, motivada por um pedido direto: o avaliador não devia precisar de um link de ativação por e-mail (que nem é enviado de verdade hoje — seção 5) para logar pela primeira vez. Em vez disso, o cadastro já atribui uma senha padrão, o avaliador loga direto com ela, e é forçado a trocá-la no primeiro acesso — com uma forma de trocar a senha novamente depois, sempre que quiser.

13.1 Decisões de modelagem

- **Login desacoplado de status_ativacao:** antes, só um Usuario com status ATIVO conseguia logar (UsuarioPrincipal.isEnabled()). Isso colidiria com o CHECK que exige comitê de todo avaliador ATIVO (seção 12), já que o cadastro de avaliador não pede comitê. Em vez de reabrir essa constraint, isEnabled() passou a liberar qualquer status diferente de INATIVO — o mesmo critério que já valia para isAccountNonLocked(). status_ativacao continua existindo e dormente para avaliadores (fica PENDENTE_ATIVACAO), sem mais nenhuma relação com a capacidade de logar.
- **precisaTrocarSenha é um sinal novo e independente:** um booleano próprio em Usuario, sem nenhuma relação com status_ativacao — só ele controla se a troca de senha é forçada.
- **O token JWT carrega o claim na emissão:** uma troca de senha bem-sucedida não altera um token já emitido, então trocarSenha() devolve um token novo (já com precisaTrocarSenha=false) em vez de só 204 No Content — o frontend substitui a sessão em memória com ele.

13.2 Migration V9

Adiciona `precisa_trocar_senha` (boolean, default false); reaproveita o "PENDENTE_ATIVACAO sem senha" para usuários legados, aplicando a senha padrão com `precisa_trocar_senha = true` onde `senha_hash IS NULL`; remove as colunas `token_ativacao` e `token_ativacao_expira_em`, que deixaram de ser usadas. Validada contra H2 (cópia adaptada em `migration-h2/`) e contra o Postgres de desenvolvimento já com dados reais — inclusive conferindo, via `psql`, que o usuário que estava pendente sem senha recebeu a senha padrão e a flag corretamente.

13.3 Backend

- `AuthService.ativarConta` (por token) foi removida; entrou `trocarSenha(usuarioId, senhaAtual, novaSenha)`, que valida a senha atual via PasswordEncoder.matches, re-hasheia, limpa a flag e devolve um LoginResponse com token novo.
- `ContaController` (novo) expõe `PUT /api/conta/senha`, autenticado para qualquer papel (cai no `anyRequest().authenticated()` do SecurityConfig — não precisa de regra nova). `AuthController` perdeu o `POST /auth/ativar`.
- `UsuarioService.cadastrarAvaliador` passou a gerar `senhaHash` a partir da senha padrão (`app.auth.senha-padrao-avaliador`, hoje `Trocar@123`) e marcar `precisaTrocarSenha=true`, em vez de gerar um UUID como token. `EmailService.enviarConviteAtivacao` foi renomeado para `enviarCredenciaisIniciais` (ainda só loga no console).
- `AvaliadorResponse.tokenAtivacao` virou `senhaTemporaria` — é o único jeito do admin saber a senha gerada, já que só o hash fica salvo.

13.4 Frontend

- `AtivarContaPage` foi substituída por `TrocarSenhaPage` (rota `/trocar-senha`), usada tanto na troca forçada quanto na voluntária — o texto da tela muda conforme `sessao.precisaTrocarSenha`.
- O JWT decodificado (`session.ts`) passou a expor `precisaTrocarSenha`; `RequireAuth` e `RootRedirect` forçam o redirecionamento pra `/trocar-senha` antes de liberar qualquer outra rota.

- Link fixo "Trocar senha" na barra lateral (`AppLayout`), satisfazendo o pedido explícito de poder trocar a senha depois, não só no primeiro acesso.
- O cadastro de avaliador (`AvaliadoresPage`) saiu de um formulário fixo no meio da tela para um modal, aberto por um botão verde "Cadastrar avaliador" no canto superior direito (variante `success` nova no componente Button, cor reservada do design system — ver seção 7) — pedido de UI à parte, nesta mesma revisão.

13.5 Verificação

Suíte de 208 testes (seção 6) passando, migration validada em H2 e no Postgres de desenvolvimento (seção 13.2), e um fluxo de ponta a ponta com navegador real (Playwright, instalado e removido só para a verificação): cadastrar avaliador → senha padrão exibida na tela → login força redirecionamento para `/trocar-senha` → troca de senha → sessão renovada redireciona para a área do avaliador → relogar com a senha nova não força troca de novo → acessar `/trocar-senha` voluntariamente pela barra lateral continua funcionando.